

Lab Assignment IV: Web Based Information System Design

Topic: Online Course Management Platform
Database Management Systems (CS39003)

Group Name: Fiery Five

Ritabrata Sarkar (23CS30045)

Akshat Priyadarshi (23CS30003)

Hritwik Upadhyay (23CS30023)

Krishnkant Sahu (23CS10035)

Harshit Singhal (23CS10025)

February 11, 2026

Contents

1	Introduction	2
1.1	Tech Stack	2
2	Database Design	2
2.1	Entity Relationship Diagram	2
2.2	Database Schema	3
2.2.1	1. Entity Tables (Core Objects)	3
2.2.2	2. Relational / Junction Tables (Connections)	3
3	Project Structure & File Descriptions	4
3.1	Directory Tree	4
3.2	Detailed File Descriptions	5
3.2.1	1. Root Configuration	5
3.2.2	2. Project Settings (course_platform/)	5
3.2.3	3. Core Application Logic (core/)	5
3.2.4	4. Frontend Templates (templates/)	6

1 Introduction

The objective of this project is to design and develop a **Web-Based Online Course Management Platform**. The system manages data related to courses, instructors, partner universities, and students. The platform features a robust backend database, a connectivity server, and a form-based web interface to facilitate interaction for four distinct user roles:

- **System Administrator:** Manages master data (Universities, Instructors).
- **Instructor:** Creates content and evaluates students.
- **Student:** Browses courses and manages enrollments.
- **Data Analyst:** Views enrollment statistics and trends.

Project Repository:

The complete source code for this project is available on GitHub:

<https://github.com/Akshat-Priyadarshi/online-course-platform/tree/main>

1.1 Tech Stack

The system implements a three-tier architecture:

Layer	Technology	Description
Database	PostgreSQL	Relational database hosted on Lab Server (10.5.18.103).
Backend	Django (Python)	Handles ORM mapping, session management, and routing.
Frontend	HTML5, CSS3, JS	Responsive UI with Bootstrap and custom scripts.
Driver	Psycopg2	Adapter for Python-PostgreSQL connectivity.

Table 1: Project Technology Stack

2 Database Design

The core of the system is a normalized relational database schema designed to support complex relationships between users and courses.

2.1 Entity Relationship Diagram

The ER Diagram below illustrates the entities (Instructor, Student, Course, etc.) and their relationships (One-to-Many, Many-to-Many).

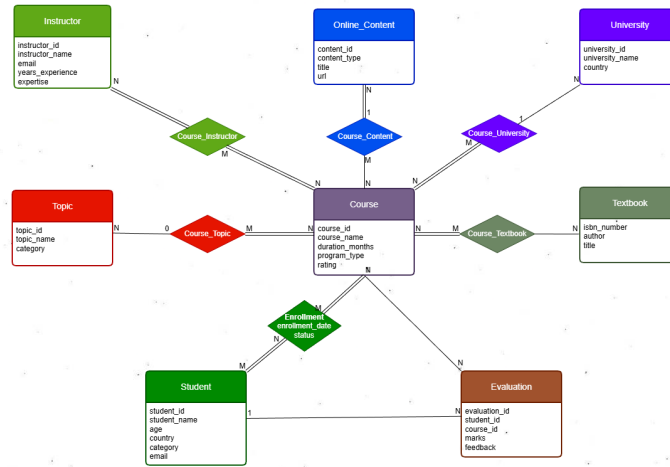


Figure 1: Entity Relationship Diagram

2.2 Database Schema

The following tables define the complete structure of the system, including all 14 tables (Entities and Junctions) as implemented in the project.

2.2.1 1. Entity Tables (Core Objects)

These tables represent the primary entities within the system.

Table Name	Attributes
University	university_id (PK), university_name, country
Instructor	instructor_id (PK), instructor_name, email, years_experience, expertise
Topic	topic_id (PK), topic_name, category
Course	course_id (PK), course_name, duration_months, program_type, rating
Student	student_id (PK), student_name, age, country, category, email
Textbook	isbn_number (PK), author, title
Online_Content	content_id (PK), content_type, title, url

2.2.2 2. Relational / Junction Tables (Connections)

These tables facilitate the relationships between the core entities.

Table Name	Attributes
Course_University	course_id, university_id (Composite PK)
Course_Instructor	course_id, instructor_id (Composite PK)
Course_Topic	course_id, topic_id (Composite PK)
Course_Textbook	course_id, isbn_number (Composite PK)
Course_Content	course_id, content_id (Composite PK)
Enrollment	student_id, course_id (Composite PK), enrollment_date, status
Evaluation	evaluation_id (PK), student_id, course_id, marks, feedback

3 Project Structure & File Descriptions

The project follows the standard **Django Project Layout**. Below is the directory tree corresponding to our specific implementation (verified via VS Code), followed by a detailed explanation of each file's role in the DBMS context.

3.1 Directory Tree

```

ONLINE-COURSE-PLATFORM2/
|-- manage.py                # [Entry Point]
|-- requirements.txt         # [Dependencies]
|-- .env                     # [Configuration]
|-- course_platform/        # [Project Settings]
|   |-- __init__.py
|   |-- asgi.py
|   |-- settings.py          # [DB Config]
|   |-- urls.py              # [Routing]
|   |-- wsgi.py
|-- core/                    # [Main Application]
|   |-- migrations/          # [Schema Versioning]
|   |   |-- 0001_initial.py
|   |   |-- 0002_alter_course...
|   |-- static/js/           # [Frontend Logic]
|   |   |-- profile_toggle.js
|   |-- admin.py             # [Admin Interface]
|   |-- apps.py
|   |-- forms.py             # [Input Validation]
|   |-- models.py            # [Database Schema]
|   |-- tests.py
|   |-- views.py             # [Business Logic]
|-- templates/               # [HTML Interface]
|   |-- admin/
|   |   |-- assign_instructor.html
|   |-- dashboards/
|   |   |-- admin.html
|   |   |-- analyst_stats.html
|   |   |-- analyst.html
|   |   |-- instructor.html

```

```
| | -- student.html
|-- instructor/
| | -- add_content.html
| | -- grade_student.html
| | -- manage_students.html
|-- student/
| | -- course_content.html
| | -- course_list.html
|-- base.html
|-- login.html
|-- signup.html
```

3.2 Detailed File Descriptions

3.2.1 1. Root Configuration

- **manage.py:** The command-line utility for the project. Crucially, this script is used to apply database changes via `python manage.py migrate`, which executes the SQL to create tables in PostgreSQL.
- **requirements.txt:** Lists dependencies. Includes Django for the framework and `psycopg2` for the database driver.
- **.env:** Stores sensitive environment variables like `DB_PASSWORD` and `SECRET_KEY`, ensuring security credentials are not hardcoded.

3.2.2 2. Project Settings (`course_platform/`)

- **settings.py:** The central configuration file.
 - **DATABASES Dictionary:** Configures the connection to the PostgreSQL server (Host: 10.5.18.103, Port: 5432).
 - **INSTALLED_APPS:** Registers the core app so Django recognizes its database models.
 - **TEMPLATES:** Points Django to the global `templates/` directory.
- **urls.py:** The main URL dispatcher. It directs incoming traffic (e.g., `/admin/`, `/dashboard/`) to the appropriate view functions in the core app.

3.2.3 3. Core Application Logic (`core/`)

This directory contains the specific logic for the Course Management System.

- **models.py (The Schema Layer):** The most critical file for DBMS. It defines Python classes that map 1-to-1 with the SQL tables.
 - Example: `class Course(models.Model)` defines the schema for the Course table.
 - Relationships: Defines `ForeignKey` and `ManyToManyField` constraints to ensure referential integrity between Students, Courses, and Instructors.

- **views.py (The Logic Layer):** Handles the application logic.
 - Data Retrieval: Executes queries like `Student.objects.get()` (SELECT) to fetch user data.
 - Data Modification: Handles form submissions to create new courses or enroll students (INSERT/UPDATE).
 - Analytics: Aggregates data for the Analyst dashboard (e.g., calculating average course ratings).
- **forms.py (The Validation Layer):** Defines HTML forms (e.g., `CourseForm`). It validates user input types (e.g., ensuring 'age' is a number) before the data attempts to enter the database, preventing errors and SQL injection.
- **migrations/:** Stores the history of schema changes. `0001_initial.py` contains the generated SQL for the initial table creation, while subsequent files track schema evolution.

3.2.4 4. Frontend Templates (templates/)

The HTML files are organized by user role to ensure separation of concerns.

- **dashboards/*.html:** Role-specific landing pages. `analyst_stats.html` is specifically designed to display the statistical output required by the project prompt.
- **instructor/manage_students.html:** Allows instructors to manage enrollments and grades.
- **student/course_list.html:** Iterates through the list of available courses fetched from the database and displays them in a grid.
- **base.html:** The master template containing the navigation bar and footer, extended by all other pages.

Execution Instructions

To deploy and run the system locally:

1. Install Dependencies:

```
pip install -r requirements.txt
```

2. Database Setup:

Ensure PostgreSQL is running, then apply the migrations to create the schema.

```
python manage.py makemigrations
python manage.py migrate
```

3. Create Administrator:

```
python manage.py createsuperuser
```

4. Run Server:

```
python manage.py runserver
```

Access the application at <http://127.0.0.1:8000/>.